Space-Efficient TREC for Enabling Deep Learning on Microcontrollers

Abstract

Deploying deep neural networks (DNNs) in a resource-constrained environment and achieving satisfactory performance is challenging. The challenge is especially difficult on microcontrollers for their stringent space and computing power. This paper introduces space-efficient TREC, a new approach to efficient DNN on microcontrollers through innovations in both computation and space optimizations. By making the optimization itself a trainable component in the target DNN, the approach maximizes the performance benefits while keeping the DNN accuracy stable. Experiments show that the solution eliminates over 96% computations in DNNs and fits them well into microcontrollers, producing 3.4-5× speedups with only marginal accuracy loss.

1. Introduction

Deep neural networks (DNNs) are in high demand from servers to the edges and now to microcontroller-based devices. Microcontrollers dominate the computing engine market for small, low-cost or energy-efficient devices [7, 17, 29, 32]. Microcontrollers exist everywhere, from household appliances [41], to cars [46], consumer electronics [51], wearables [50] and so on. It is estimated that 250 billion microcontrollers are already in use [49]. Microcontrollers are very resource-constrained. For example, STM32 F469I contains only 324KB SRAM and 2048KB flash on-chip memory [8]. As a result, they have been widely regarded viable only for simple applications (e.g., keyword spotting [36, 60]), rather than complex DNN models.

But the demands for DNNs on microcontrollers [4, 14, 17, 29] keep growing, for three reasons. First, more advanced AI can become possible on microcontrollers, and hence expend the range of applications. Second, executing large DNNs locally on microcontrollers is essential for reducing energy consumption while increasing performance efficiency [8]. By eliminating the need for streaming from edge to the cloud [24], application latency can also be reduced, while overcoming network congestion issues [23]. Finally, such designs eliminate many privacy concerns, as all user data is processed locally [25].

We show a use case of anomaly detection in Figure 1, which applies microcontrollers on a servo motor to retrieve vibration data [52]. With the help of microcontrollers, anomalies can be detected and reported in real time. In this particular case, the detection accuracy of 0.67 and a latency of over 1 second [13] are improved to 0.82

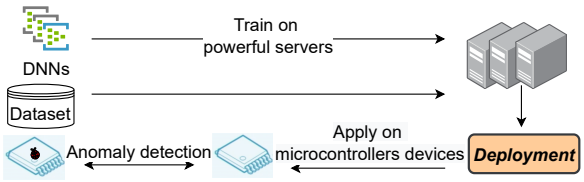


Figure 1: Use case of anomaly detection.

and 317ms when DNNs are moved from cloud to the device end [37]. In the same vein, many other tasks can benefit from on-device AI, ranging from exoskeletons [22], to voice activation [26, 56], object detection [5, 6, 48], and so forth.

There are some prior efforts trying to enable DNNs on microcontrollers, by creating light-weight neural networks [30, 57] and other designs [11, 15, 45]. However, the available DNNs that can be executed on microcontrollers are still very limited, and those that can run are often subject to long latency [35, 55].

Challenges to efficient DNN on microcontrollers lie in how to mitigate the tensions between performance and limited resources, shown in three aspects: (i) how to minimize the long execution time of DNN inferences under limited computing resources; (ii) how to host complicated DNNs in the limited memory including its inputs, outputs, weights, activation maps, and intermediate results; (iii) how to establish an optimal setting for minimizing the accuracy loss when optimizing the DNNs.

In this work, we propose a new approach, *space-efficient TREC*, to substantially improve the state of the art of DNNs on microcontrollers. TREC stands for *transient redundancy elimination-based convolution*. Its key idea is to identify similar elements in the input data of a convolution layer and avoid repeating similar computations on the fly. Because input data changes across inferences, the redundancy that TREC exploits is transient, complementary to the kind of (lasting) redundancy in DNN weights that have been exploited by the large body of work on DNN compression.

Specifically, we have made progress in three ways. First, by incorporating a novel TREC operator into our system, we successfully group similar neuron vectors into clusters to detect redundant computations and direct clustering results back to the DNN. By adopting this design, we also significantly reduce the volume of computations for the entire network. Second, as the new operator introduces additional space overhead, for space efficiency, we design a kernel reuse technique to reduce the space for storing parameters in the newly-built network. Moreover,

we embed a two-step stack for storing clustering ID in TREC and use a reversed index to help locate the entries in the stack. Third, with the systematic design, finding the best configuration becomes part of the model training process, seamlessly integrated as a component of the proposed architecture, thereby substantially reducing the accuracy loss.

We evaluate our solution by applying it to three popular DNN networks, namely CifarNet [27], ZfNet [59], and SqueezeNet [20], on two microcontroller models. Our experiments show that, by implementing our solution on microcontrollers, we are able to avoid over 96% computations on convolution layers, and achieve an average of 3.4-5× reduction in the overall network latency with no or marginal accuracy loss.

Several previous studies have exploited similarities in inputs for DNNs. They either rely on special hardware [44] or use random hashing vectors that cause unstable inference accuracy [38]. None of them target microcontrollers or deal with the stringent space limitation. To our best knowledge, the solution from this work is the first that proposes trainable TREC and welds it into DNN in a space-efficient manner for microcontrollers.

The main contributions of our work are as follows:

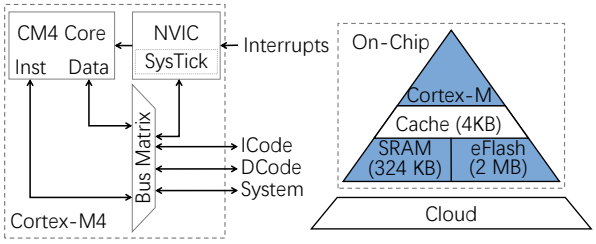
- We incorporate the use of TREC as a new solution to reduce computations in DNNs running on microcontrollers.
- We introduce a set of optimizations to mitigate the space overhead incurred by TREC.
- We empirically evaluate the effectiveness of the new solution on two models of microcontrollers, confirming the substantial benefits of the new solution in enabling efficient DNNs on microcontrollers.

2. Background and Related Work

In this section, we introduce the deployment of neural networks on microcontrollers, the work related with input reuse and work related with model compression.

Microcontrollers. Microcontroller (MCU) is an energy-efficient processor that is ubiquitous in our lives. We show an example of MCU architecture and its memory hierarchy (STM32F469I in our case) in Figure 2. Figure 2 (a) shows that the Cortex-M4 core architecture consists of a 32-bit processor (CM4) and a small number of critical peripherals. The CM4 core is a Harvard-architecture, which means that it utilizes distinct interfaces to fetch instructions (Inst) and data (Data). This helps ensure that the CPU does not run out of memory, as it enables simultaneous access to the data and instruction memories. The special feature that distinguishes Cortex M4 from CM3 [10] is that CM4 includes, for the processor, single-instruction multiple-data (SIMD) extensions that are one of the most effective approaches to achieving faster arithmetic compute performance in the CPU context. From the CM4's perspective, everything appears to be memory – it distinguishes only between instruction fetches and data accesses. The Cortex-M4 communicates with manufacturer-specific hardware via three memory buses – ICode, DCode, and System – each of which is configured to access a distinct section of memory.

Figure 2 (b) shows its on-chip memory hierarchy consisting of very limited memory space. We can see that microcontrollers are typically comprised of a central processing unit (CPU), cached memory for frequently accessed data, static random-access memory (SRAM), and an on-chip flash memory for storage.



(a) Architecture of Cortex-M4. (b) Memory Hierachy for MCUs.

Figure 2: An illustration of the architecture and memory hierarchy for microcontrollers.

Microcontrollers are especially energy efficient with low-power (0.166W for F469I board) and cost effective (around $10) compared to common processors like CPU and GPU. Given this, microcontrollers provide very limited computing resources and a limited volume of storage (around 1 cm^3) that developers need to take care of.

Work on input reuse. Several studies have explored similarities in inputs for DNN accelerations. *Deep reuse* [38, 53] is a pure software approach. It inspired this current work. To compute the convolutional layer, the common practice is to unfold the input images and filters into input matrix x and weight matrix W, and then perform General Matrix Multiplication (GEMM) with two matrices, as shown in Figure 3. The idea behind deep reuse is that strong similarities exist among neuron vectors. Here, a neuron vector is composed of several consecutive elements in a row of the unfolded input matrix x. Therefore, the neuron vectors can be clustered into a small number of groups, and the computation results for the cluster centroids can be reused by all neuron vectors in the same cluster. The work uses some random locality-sensitive hashing vectors for the online clustering. It does not consider space efficiency; it actually increases the space usage substantially due to the extra space needed to store hashing vectors and cluster indices. The use of random hashing vectors also cause large fluctuations in the inference accuracy (detailed in Section 7). Some other studies [44] have designed special hardware to exploit input reuse.

Relations with model compression approaches.

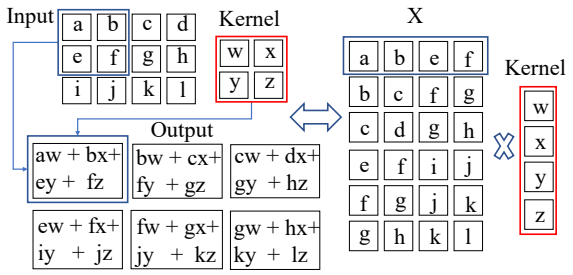


Figure 3: An illustration of convolution in *deep reuse*.

Many studies on DNN compression [18] have exploited the redundancy among DNN parameters, which is orthogonal to input-level reuse. Examples include quantization [58], squeezing filter size [12], conducting feature compression to activation map [47], adding a downsampling [43], converting large models to light-weight counterparts through knowledge distillation [19], and applying adaptive LSH framework to reduce the size of model updates like Mongoose [9]. These techniques can significantly reduce the model size (i.e., weights and biases), the amount of computations, and data required to stream from edge to the cloud. For resource-stringent microcontrollers, input reuse and model compression have to be applied at the same time as shown later (TREC design in Sections 4&5 and model compression in Section 6) in this paper.

3. Overview

Running large DNN inference on microcontrollers is especially challenging because of the strict resource constraints both in terms of computing capabilities and memory footprint. Removing computation redundancy and achieving space efficiency is thus essential for enabling efficient DNN deployment on microcontrollers.

Redundancy in DNNs can be categorized, based on where the redundancy originates, into *lasting redundancy* and *transient redundancy*. Lasting redundancy originates from the model parameters. As parameters are unchanged when applying inference, this kind of redundancy can be eliminated by methods such as pruning and quantization. Transient redundancy, however, arises from input data or activation maps, taking the form of similar tiles within an input image or a single activation map.

There are many techniques that can be adopted for the elimination of lasting redundancy when running neural networks. For example, CMSIS-NN applies quantization to the kernel weights and consequently avoids floating point computations for DNN networks, thus improving inference performance. However, ways of reducing transient redundancy on microcontrollers remain insufficiently explored.

To address this issue, we devise a principled way to detect and remove transient redundancy for DNNs on microcontrollers. By incorporating the novel operator called Transient Redundancy Elimination-based Convolution (TREC) into DNNs, they automatically detect and remove the transient computation redundancy in a network. The proposed architecture has multiple benefits. First, by eliminating transient redundancy, TREC minimizes the computation volume, thus bringing the maximum computation-elimination benefits and improving the performance of the entire network. Second, TREC is compatible with both training and inference tasks, allowing for a simple plug-and-play replacement of convolutional layers in standard convolutional neural networks (CNNs) for training and inferencing. This approach guarantees highly robust model accuracy. Third, applying TREC in the model is orthogonal to methods targeting removal of lasting redundancy, which can bring further benefits when combined together. The proposed architecture is described in detail in Section 4.

Adding the new operator in the DNN inference stage reduces the amount of required computations significantly, but it also introduces space overhead, as additional space for LSH matrix and clustering containers are required. To achieve space efficiency, we conduct *two-step stack substitution* for the DNN network to minimize the space occupancy for the clustering containers. Furthermore, for extreme cases where efficient space usage is of paramount importance (e.g., microcontrollers), we propose the *kernel reuse* technique to remove the LSH matrix overhead while still eliminating the transient redundancy when performing DNN inference. In conclusion, the overview of space-efficient TREC is outlined in Figure 4.

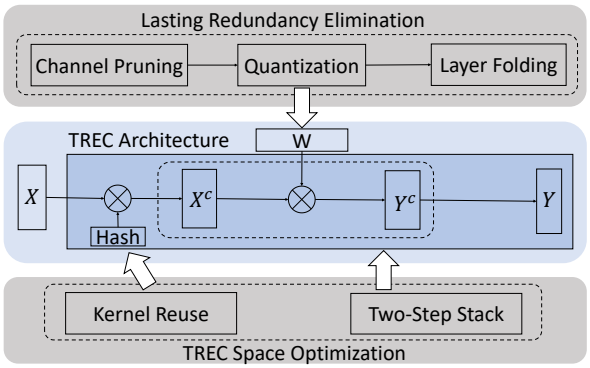


Figure 4: Overview of space-efficient TREC.

4. TREC Architecture

In this section, we first offer some information on TREC and delineate the key conditions for its use in DNNs, before describing in detail the TREC design aimed at DNN deployment on microcontrollers.

4.1. Architecture Overview and Key Conditions

The main aim of TREC is grouping similar neuron vectors into clusters and then redirecting clustering results back to the DNN. To achieve this objective, it fuses the detection and avoidance of transient redundancy into DNN, making them part of its inherent architecture [2].

As illustrated in Figure 5, the original GEMM convolution is $\mathbf{y} = \mathbf{x} \cdot \mathbf{W}$. The TREC operator can be decomposed into four steps. First, the input matrix $\mathbf{x}$ passes through a clustering component. At a high level, TREC uses Locality-Sensitive Hashing (LSH) to do the clustering. In this example, $\mathbf{x} \in \mathbb{R}^{4\times 4}$ is first sliced into two 4×2 sub-matrices, each of which has its own hash functions. And the 4 row vectors in each sub-matrix are grouped into two clusters. In the second step, for each $\mathbf{x}$, $\mathbf{x} \cdot \mathbf{W}$ is performed using the representative neuron vectors, allowing the matrix size for computing to be significantly reduced. In the third step, for each sub-matrix, vectors in the same cluster use the computation results from the corresponding centroid neuron vector to recover the full-sized output matrix. Finally, the final result is obtained by adding the results computed separately for each sub-matrix.

The main computation savings come from the clustering component. This is because in GEMM-based convolution, we should compute four results for each neuron vector. By using TREC, we use the result of the centroid computation in place of the per-row results for each of the row vector. Therefore, the number of vector-matrix computation reduces from four to two.

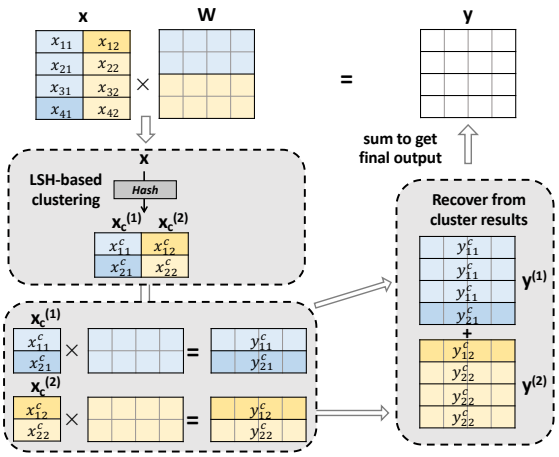


Figure 5: An illustration of TREC.

Transient redundancy elimination benefits and the key condition. By grouping neuron vectors into clusters, the size of the input matrix is significantly reduced, allowing for low computational complexity for the subsequent matrix multiplication. Consider that an input matrix X for GEMM (after im2col) is of $N \times K$ dimension and a weight matrix W is $K \times M$ size. The clustering step can be abstracted as applying a hash function matrix $Hash$ to X. Let $K \times H$ be the dimension of the hash matrix. We can assume that the neuron vectors can be grouped into N_c clusters. For each input image or activation map, the benefits for removing the *transient redundancy* can thus be measured by a *redundancy radio*, i.e., $r_t = 1 - \frac{N_c}{N}$ as the reduction of input size. N is the total number of neuron vectors, and the number of centroid vectors is equal to the number of clusters, namely N_c. The number of required computations is thus reduced to N_c. We can conclude that r_t indicates the fraction of *transient redundancy* within input images or activation maps.

Using this measure, the total Floating-point Operations (Ops) for a conventional GEMM-based convolution is $N \cdot K \cdot M$, while the Ops for TREC will be $(\frac{H}{M} + 1 - r_t) \cdot N \cdot K \cdot M$, since TREC also involves a $Hash$ matrix multiplication. Therefore, in order for TREC to expedite DNN inference (i.e., $(\frac{H}{M} + 1 - r_t) \cdot N \cdot K \cdot M < N \cdot K \cdot M$), the following *key condition* must hold: $\frac{H}{M} < r_t$. As shown in the experiments, the average transient redundancy elimination benefits r_t exceeds 96%.

Comparison with *deep reuse*. Prior studies like *deep reuse* have treated *transient redundancy* in an ad-hoc manner. *Deep reuse* takes place as extra operations outside DNN, using LSH with random hashing vectors for online data clustering. Such ad-hoc treatment causes severe uncertainty about the impact of the clustering errors on the DNN performance. Experimental results show that *deep reuse* causes significant (e.g., 5%) fluctuations on DNN accuracy.

Incorporation of TREC into our system involves comprehensive understanding to the fundamental aspects of *transient redundancy*. There are many underlying differences between TREC and the *deep reuse* technique. First, the newly introduced DNN operator TREC incorporates transient redundancy detection and avoidance directly into DNN's inner architecture, while *deep reuse* functions outside the DNN. Second, based on this design, we are able to sustain a consistent and powerful inference performance with transient redundancy removal through iterations. *Deep reuse*, however, suffers from unstable inference performance across runs. Third, TREC mitigates the interference to the converged training results introduced by transient redundancy elimination. *Deep reuse*, on the other hand, cannot guarantee a converged training.

4.2. LSH-based Clustering

LSH (Locality Sensitive Hashing) has been identified as the method for the clustering module among many others because it costs low overhead. For each input vector $\mathbf{x}$, a hash function h is determined by a vector $\mathbf{v}$ using the expressions given by Equation 1:

$$h_{\mathbf{v}}(\mathbf{x}) = \begin{cases} 1, \; if & \mathbf{v} \cdot \mathbf{x} > 0 \\ 0, \; if & \mathbf{v} \cdot \mathbf{x} \leq 0 \end{cases} \tag{1}$$

In our design, we use H hash functions. So given an input vector, LSH maps it to a H-bit vector. Nearby input vectors often produce the same bit vector after hashing, and the number of hash functions, H, adjusts clustering roughness.

The bit vector's decimal integer value can be utilized as a cluster ID after applying LSH. Therefore, the neuron vectors with the same cluster ID can form a cluster and reuse the computing results from the centroid vector in place of the individual results for each of the vector.

4.3. Training TREC

So far we have discussed how TREC can benefit from reusing the centroid results. We have also shown how TREC performs efficient convolutions in the following stages: 1) clustering: grouping similar neuron vectors through hash functions, 2) computing and reusing results from centroid vectors in each cluster in place of the results of each vector in the cluster, and 3) recovering back to final results. Now, we explain how to jointly learn the hash functions and kernel weight parameters, $Hash$ and W. Direct learning of TREC leads to problems: 1) discrete mapping problem, where we need to map values to 0 or 1 according to their signs, and 2) combinatorial optimization problem, where we have to compute the centroids for each cluster. To get around this, we reformulate the clustering stage.

We propose the following workflow for clustering in Figure 6 to make back-propagation work with the clustering in TREC. First, to solve the discrete mapping problem, after the input matrix $\mathbf{x}$ is multiplied by the $Hash$ matrix, the projected matrix is subjected to an element-wise sigmoid functioning as a binary classifier. In line with the proposed design, each neuron vector is thus transformed into a H-bit vector. In the next step, the bit vector is converted into a cluster ID, akin to converting a numerical value from its binary representation to the decimal counterpart. We then apply the mapper matrix to the ID vector and, after applying a Gaussian function, we obtain a bitmap of the neuron vector where each column indicates whether this vector belongs to a corresponding cluster. This bitmap helps to transform the computation of centroids from a combinatorial problem to a matrix multiplication.

After obtaining centroids, we can multiply the centroids with the weights W and perform the rest of the computation step of TREC as in Figure 4.

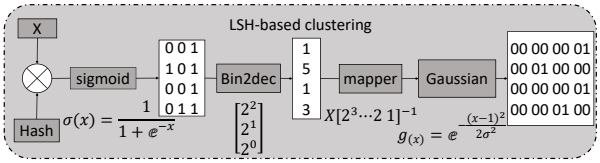


Figure 6: Training design for the clustering component in TREC.

4.4. Properties of TREC

We next discuss the correctness of TREC, and why it is robust and maintains convergence. Detailed proofs are available in the report [2].

Correctness. Reusing results from the centroid vector for the vectors in the whole cluster is driven by the similarities among the vectors in the cluster. The learning process in TREC helps minimize the approximation errors. If we train the neural network with *random* LSH vectors, i.e., without learning the *Hash* matrix in the back propagation, accuracy could drop sharply. The *learned Hash* matrix in TREC makes a big difference in improving the accuracy of the network, as our experiments in Section 7 will show. The lesson is that the *Hash* matrix is learned and updated in a way that it resonates with the clustering process. In a sense, it chooses the optimal LSH vectors that are able to detect the similarities between vectors, and it helps group the similar vectors into the same cluster.

Robustness. Traditional GEMM-based CNNs are robust in that they possess stability for small input perturbation. In order to assess the robustness of a neural network, we apply the *Lipschitz constant* L to analyze TREC. L gives the relations between the input perturbation ε and the output variations δ, namely $\delta \leq L\varepsilon$. The *Lipschitz constant* for TREC is rigorously proved to be bounded by the GEMM-based Convolution, namely, $L(TREC) \leq L(Conv)$. Therefore, when the input image suffers from small perturbation, according to the properties of *Lipschitz continuous* of TREC, the output variation, denoted as δ_{TREC}, is bounded by the L_∞-norm of the weight matrix W, i.e., $\delta_{TREC} \leq L(TREC) \leq L(Conv) \leq \|W\|_\infty \varepsilon$. This shows that TREC is robust to input perturbation.

Convergence. TREC-equipped DNNs converge under reasonable assumptions: (a) The objective function $F(W,H)$, in which W and H correspond to the weight matrix and *Hash* matrix, is continuously differentiable, and (b) $F(W,H)$'s partial derivatives ∇F are *Lipschitz continuous* where their first and second moments meet certain limits. Given the above assumptions, we are able to obtain the fact that the expected change in F between two iterations are bounded. Therefore, for stochastic gradient descent optimization strategy, the partial derivatives of the objective function F converge to zero after many iterations. This is true to both fixed and diminishing stepsizes and thus proves that we have found the locally optimal solution.

4.5. TREC with Sparse Matrices

TREC is applicable in the presence of sparse matrices. In this part, we explain how that can be done with both sparse weights matrix and sparse input matrix.

Sparse weights matrix. One typical case where sparse matrix shows up is when weight pruning is used. We will draw on PatDNN [40] as our example. PatDNN prunes a DNN kernel via patterns. The left graph in Figure 7 shows a 4-element pattern left after a 3x3 kernel is pruned. In PatDNN, the sparse weight matrix is encoded in a Filter-

Kernel-Weight (FKW) format [40]. When a convolution kernel slides through an image or activation map, only the pixels within the pattern's frame contribute to the output. In the example in Figure 7, only four elements in the input are used in the dot-product with the kernel in each step. TREC can be applied by regarding the four elements used in each step as a vector for clustering. If two of such vectors are similar and fall into the same cluster, TREC avoids the repeated dot-product calculations.

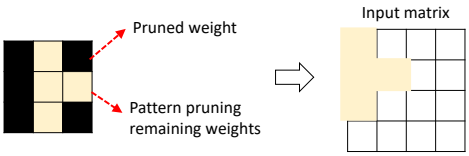


Figure 7: Illustration of a sparse pattern kernel.

Sparse input matrix. We next look at how TREC can be applied if the input matrix is sparse. We discuss it in two scenarios. In the first scenario, the original algorithm simply treats the sparse input in the same way as it treats a dense input. Clearly, TREC can be applied as we have already described, and because there are many zero vectors, TREC would cluster them together and give significant speedups. In the second scenario, the original algorithm already tries to do calculation only on the non-zero parts of the input. *Sparse convolution reorganization* is one way to do that [16, 34, 54]. It groups non-zero elements together to form a dense matrix. This matrix is then multiplied with reorganized kernel matrices. In this way, it avoids the inefficiency of frequent irregular accesses. TREC can be applied to the matrix after the reorganization. We provide details in Appendix A and report the experimental results in Section 7.5.

5. Achieving Space Efficiency

5.1. Space Pressure For DNNs on Microcontrollers

In this section, we analyze the space pressure for DNNs on microcontrollers. Unlike desktop systems, microcontrollers have a rather flat memory system, as they are equipped with on-chip main memory only. To fit large DNNs into such a memory system, given the small size of microcontroller devices, we carefully analyze the space usage for the original network and the additional space required for TREC.

Figure 8 provides a typical memory breakdown for neural networks on the STM32F746ZG SRAM. The runtime overhead for the mbed-os is fairly small, requiring just 25 KB on SRAM. Loading buffers are needed for image input and are allocated. As CifarNet is equipped with only 3 convolutional layers, so the full weights and biases are stored and buffered on the persistent buffers, where pointers to the intermediate buffers are also stored. Activation maps are also allocated on SRAM as well.

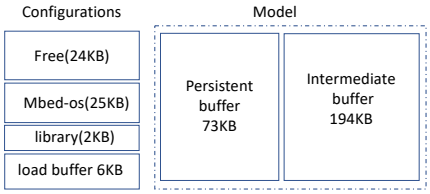


Figure 8: Breakdown of SRAM memory for an original DNN without TREC on STM32F746ZG.

Additional space for TREC. Adding the TREC operator to the DNN can minimize the required computations for the whole network, allowing for efficient inference performance. However, it brings additional space overhead. Specifically, an extra *Hash* matrix and containers for holding cluster information are needed. Since there are at most 2^H different clusters, the additional space overhead for clustering computing is $O(L \times 2^H)$, space required to be allocated on the *free* section in Figure 8. Consider TREC operators in N layers, the total space overhead for the *Hash* matrix, however, is $\Sigma_{n=1}^{N} C_{i-1} \times k_i^2 \times H$, where k_i and C_i denotes the kernel size and the channel size of layer i, respectively.

Empirical analysis for space overhead. Space resource is very limited on microcontrollers. Specifically, SRAM is important as primary storage because direct computations are performed on it. Flash memory stores read-only data and uses SRAM to load data when doing computation. However, applying TREC can bring additional burden to SRAM spaces. Table 1 shows the SRAM size of four typical STM32 microcontrollers and the space ratio of TREC overheads to these boards. The corresponding L and H configurations for determining the TREC space overhead are displayed in Table 4 in Section 7.3. For each DNN network, *Hash* denotes the additional space required for the LSH matrix, while *Cluster* relates to the indexing and storing vectors for each cluster. For example, TREC requires an additional 16KB to conduct clustering operations and 41KB for hash tables for SqueezeNet.

Table 1: Space overheads introduced by TREC for the three networks and their ratio to the SRAM of four boards.

Network		CifarNet		ZfNet		SqueezeNet	
		Hash	Cluster	Hash	Cluster	Hash	Cluster
Space overhead		20 KB		40 KB		56 KB	
Breakdown		60.6%	39.4%	65.8%	34.2%	91.1%	8.9%
Space ratio	F205 (64KB)	31.3%		62.5%		87.5%	
	F303 (80KB)	25.0%		50.0%		70.0%	
	F446 (128KB)	15.6%		31.3%		43.8%	
	F746 (320KB)	6.3%		12.5%		17.5%	

Space limitations.

Original networks have large model sizes and need *lasting redundancy* elimination to fit in the microcontroller board. For example, the sizes of ZfNet and SqueezeNet are 5MB, which are reduced to below MB after pruning. TREC affects the footprint of the pruned model.

In fact, Table 1 shows that TREC space overheads are substantial compared to the total SRAM capacities on microcontrollers. Therefore, the space overhead introduced by TREC is non-negligible and hinders running DNN applications on microcontrollers, considering facets such as larger batch sizes.

5.2. TREC Space Optimization

Although only trivial KB-level space occupancy overhead is introduced, which is negligible in the context of cloud computing, this capacity requirement poses challenges in the materialization of TREC on microcontrollers. Achieving space-efficient TREC is hence crucial.

Conventional optimization. Conventional implementations for space savings include DNN channel pruning, quantization, and layer folding. For these techniques, the goal is to prune model parameters and reduce the required static space for storing the model structure and parameters. Therefore, these *offline* pruning methods rely on delicate detection and examination of unnecessary parameters in the network, aiming at removing lasting redundancy. These techniques are discussed as implementation details in Section 6.1. Another category of potential space savings lies in reducing the *online* dynamic execution space occupancy of TREC, namely space savings for the clustering component. Emphases are placed on the *Hash* matrix and the choice of data structure facilitating storing and indexing vectors.

TREC optimizations. In the following, we propose two techniques for achieving space-efficient TREC. The first idea is to implement hashing in a space efficient way by re-using parts of a matrix that is already part of the DNN computation as the collection of vectors required to implement clustering. Vectors making up this matrix thus serve "double duty" as hashing vectors and matrix elements. The technique avoids huge memory overheads and achieves high speedups on real DNN computations.

For the second design, we embed a two-step stack for storing clustering ID in TREC and use a reversed index to help locate the entries in the stack. We will go through these two techniques in further depth.

5.3. Kernel Reuse

As discussed in Section 5.1, vanilla TREC increases the space usage due to extra space needed to store hashing vectors. We hereby propose *kernel reuse* as a technique to address the space issues for the *Hash* matrix on micrcocontrollers.

To achieve space efficiency, the idea of *kernel reuse* is to implement hashing by reusing parts of the weight matrix as the collection of vectors required to conduct clustering.

A closer look at the weight matrix. For a GEMM convolution, each filter slices through the input matrix as illustrated in Figure 9. There are six filters in total, and each filter composes of one 3×3 kernel. After im2col, we focus on the weight matrix. Each filter is expanded to a column vector. Since there are six filters and nine elements in each filter, the weight matrix has a 9×6 size. Now, we consider the clustering step. Suppose we set $H = 3$, namely, the hashing comprises of three vectors for clustering. Immediately, we find it possible to take advantages of the first three columns of the weight matrix W and reuse them as the *Hash* matrix for clustering the rows of the X matrix. While computations have to be performed on each column of W, the partial results from the first three columns thus serve double duty as matrix outputs and hashing outputs. This method avoids huge memory overheads and achieves time savings.

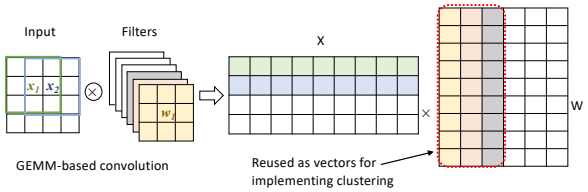


Figure 9: A closer look at the weight matrix.

Detailed design. A detailed workflow of the proposed technique is illustrated in Figure 10. First, we reuse parts of the weight matrix as *Hash* matrix. Next, we multiply X with *Hash* and obtain a projection matrix. Based on this result, we project each row vector to a cluster. Specifically, for each row, i.e., a neuron vector $\mathbf{x}$, we apply $h_{\mathbf{v}_i}(\mathbf{x})$ for all $\mathbf{v}_i, 0 \leq i < H$. In the end, the cluster ID of a neuron vector is encoded as a H bit integer $clusterId(\mathbf{x}) = (h_{\mathbf{v}_0}, h_{\mathbf{v}_1}, ...h_{\mathbf{v}_{H-1}})$. Vectors with the same cluster ID are grouped in the same cluster. Third, we compute a centroid vector for each cluster and compile them in a centroid matrix. Finally, we multiply the centroid matrix with the matrix and obtain the result for each cluster.

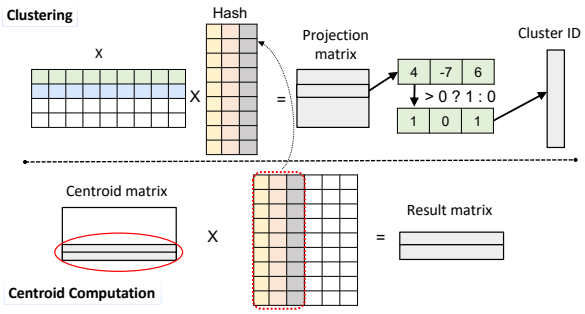


Figure 10: Illustration of kernel reuse.

Solving the dilemma for *kernel reuse* training. Training TREC with *kernel reuse* is one key challenge. *Kernel reuse* seeks ways to embed LSH vectors in the weight matrix. However, TREC requires separately updating

Hash and the weight matrix W in each training iteration, which leads to inherent discrepancies between *Hash* and W. To get around this, we embed the LSH vectors in the weight matrix by binding *Hash* to the columns in W and sharing the gradients in each iteration. As illustrated in Figure 11, in the back propagation, instead of updating LSH vectors and the weight matrix independently, the gradients for *Hash* are copied directly from the gradients of the corresponding positions in the weight matrix. The *Hash* matrix is thus embedded in the weight matrix after each iteration.

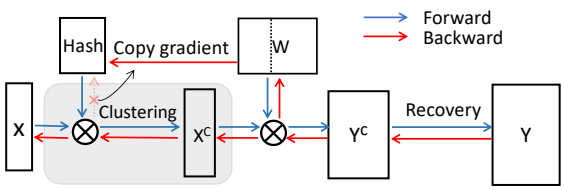


Figure 11: Illustration of *kernel reuse* training.

Maintaining properties of TREC. Another factor to note for building *kernel reuse* based TREC is to understand the impact of *kernel reuse* on the correctness, robustness, and convergentability of TREC.

Correctness. As stated above, *kernel reuse* introduces errors in clustering because gradients are directly copied from those of the weight matrix. This error is then passed on to the next forward propagation and got corrected in the back propagation. Through training, *kernel reuse* minimizes errors while saving space.

Robustness and Convergence. *Kernel reuse* does not change the *Lipschitz constant* for TREC, and thus it keeps the robustness of TREC. The convergence of *kernel reuse* can be proved following the same steps as those of TREC by viewing the objective function F as a function of W, namely, $F(W)$.

Space savings. *Kernel Reuse* removes the space overhead from *Hash* matrix and greatly saves space. This space saving does not sacrifice the expressive functionality of LSH to cluster vectors. This is because LSH vectors are kept to be mutually independent even by reusing parts of the weight matrix. Since filters are trained independently in the weight matrix, columns of the weight matrix are mutually independent. This guarantees that LSH vectors are independent from each other.

5.4. Two-step Stack Substitution

General design. Another part of space overhead of TREC comes from the data structures for indexing vector entries. The space constraints on microcontrollers require efficiently storing and accessing neuron vectors in different clusters while minimizing the space occupancy. Therefore, with the help of a reversed ID table and a stack, a representative vector is stored in each cluster.

Detailed design. Suppose now we have each vector's *cluster* ID. The goal is to do clustering by grouping the vectors with the same ID together and finally get a centroid matrix containing the centroid vector for each cluster. However, storing vectors into groups according to cluster IDs and computing the centroids for each cluster is not space efficient. This is because pre-allocating fixed-size arrays for each cluster poses too much burden on the memory. To make it more space efficient, we form the centroid matrix in a streamlined way.

At any time, the space overhead is only a representative centroid vector for each cluster. These centroid vectors are stored in a stack with an index table for entry access. After finishing processing, these centroid vectors form a centroid matrix, and we can multiply it with the weight matrix. We then use the result of the centroid computation in place of the per-vector results for each vector in the cluster. A workflow sample is shown in Figure 12.

Each neuron vector is projected to a cluster ID Id. When it is the first vector with cluster ID = Id, we add this vector in the buffer stack and store a pointer to it in the *reverse ID table*. In fact, the x-entry in the reverse ID table stores the pointers, if existed, to the buffer stack for the cluster with $Id = x$. When another vector $\mathbf{v}_{new}$ with the same cluster Id arrives, we can directly find the representative centroid vector $\mathbf{v}_{old}$ in the stack and add the weight of this vector to it. Formally, if $count_x$ denotes the number of vectors in cluster x, we have

$$\mathbf{v}_{new} = \frac{\mathbf{v}_{old} \times count_x}{count_x + 1}, count_x = count_x + 1 \qquad (2)$$

There are at most 2^H (Id ranges from 0 to $2^H - 1$) clusters, thus at most 2^H representative vectors in the stack. Finally, the centroid vectors form a centroid matrix.

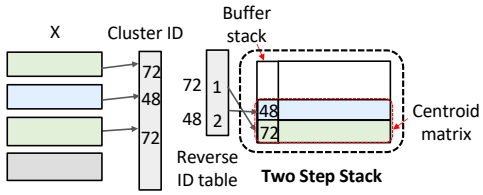


Figure 12: Stack for the representative vectors.

For each representative vector, we multiply it with the weight matrix to obtain a result vector, so that the vectors in the same cluster can reuse the result to speedup the computation. Note that the representative vectors stored in the stack actually form a matrix. Therefore, we only have to multiply the centroid matrix by the weight matrix, and the result vectors can be retrieved as the corresponding row in the result matrix.

6. Implementation

We discuss the general workflow of our deployment and other implementation details in this section.

6.1. Lasting Redundancy Elimination

To load a large DNN model onto Microcontrollers, the first steps are to eliminate lasting redundancy through *offline* methods so as to reduce the size of the model. These include an effective pruning to reduce the number of parameters in a model when performing inference (detailed in Section 6.1), quantization of transforming 32-bit floating point data into 8-bit integer (detailed in Section 6.1), and the use of batch norm layer folding into the convolution layer that proves efficiency and effectiveness (detailed in Section 6.1).

Parameterized model Pruning. The first and foremost method of reducing the size of a network is model pruning. An effective pruning method should first reduce the number of parameters in the network significantly and, at the same time, easy to maintain the architecture structure of the model.

Unstructured pruning. We apply PyTorch pruning [42] first and find huge potentials in reducing the size of the model. Specifically, this default pruning method masks partial elements of weight matrices thus converting these dense matrices into sparse matrices, so the network is transformed into a sparse network accordingly. That is to say, the entries with small values in the weight matrix will be set to zero. However, the sparse tensor operations are currently not supported for PyTorch. Therefore, we cannot see the acceleration effect this sparse network can achieve. Fortunately, we do benefit from such analysis; as an exploratory experiment, over 90% of the entries in a weight matrix can be masked without harming the accuracy of the whole network. This result shows huge potentials in reducing parameters, particularly the weight matrices, of the network.

Structured pruning. In this work, we identified torch-pruning as the pruning method. Torch-pruning is a widely used pruning method [3] and is special in that, instead of trying to mask certain entries in a weight matrix, it removes the whole channels from neural network for acceleration.

Network quantization. Floating point weights and activations are used to train neural networks. Previous research [31] has shown that fixed-point weights are enough for executing neural networks with minimum accuracy loss. Because microcontroller devices have limited memory, the quantization of 32-bit floating point weights to 8-bit fixed point weights for deployment reduces the model size by a factor of four. Furthermore, fixed-point integer operations in common microcontrollers are substantially faster than floating point operations, which is another justification for executing quantized models during deployment.

Method. Assuming a fixed-point format and scaling by twos, the represented value is $A \times 2^n$, where A denotes the integer value and n is the radix point's location. Due to the power-of-two scaling, the scaling factors for the bias and outputs are passed to the kernels as parameters. Because ARM Cortex-M CPUs can lack a dedicated floating-point unit (FPU), we utilize this sort of quantization instead of TensorFlow's 8-bit quantization.

Batch norm layer folding. When we apply quantization, problems arise for *batchnorm* layer, since the original floating point means that μ and standard deviation σ are required. To solve this problem, we fold batch norm into the convolution when performing inference for the batch norm layers in DNNs.

In batch norm folding, we replace the convolution followed by a batch normalization with just one convolution with different weights. This would remove the precision loss if we quantize μ and σ from 32 bit floating point to 8 bit integer. Furthermore, it reduces the number of operations to be performed at inference time, thereby speeding up the entire network.

Specifically, the convolution operator followed by a batch normalization can be expressed, for an input $\mathbf{x}$, as:

$$\begin{aligned} \mathbf{z} &= W * \mathbf{x} + \mathbf{b} \\ y_i &= \gamma \cdot \frac{z_i - \mu}{\sqrt{\sigma^2 + \varepsilon}} + \beta \end{aligned} \tag{3}$$

When we rearrange the value of W and $\mathbf{b}$ such that:

$$\begin{aligned} W_{\text{folded}} &= \gamma \cdot \frac{W}{\sqrt{\sigma^2 + \varepsilon}} \\ b_{\text{folded}} &= \gamma \cdot \frac{b - \mu}{\sqrt{\sigma^2 + \varepsilon}} + \beta \end{aligned} \tag{4}$$

We manage to remove the whole batch norm layer and obtain the same result with only one convolution.

6.2. Meeting Accuracy Requirements

The input matrix X is divided into three smaller matrices and partial results are computed as $\mathbf{y}^{(1)}, \mathbf{y}^{(2)}$, and $\mathbf{y}^{(3)}$. For the normal full neuron vector granularity in microcontrollers, $\mathbf{y}^{(i)}$ is truncated and stored in an 8 bit integer. With smaller granularity, all $\mathbf{y}^{(i)}$ should be kept in 32-bit containers when the result $\mathbf{y} = \mathbf{y}^{(1)} + \mathbf{y}^{(2)} + \mathbf{y}^{(3)}$ is added up with all these partial results. After that, $\mathbf{y}$ gets scaled and truncated.

6.3. Implementing TREC

TREC design is at the core of our work in enabling efficient DNNs on microcontrollers. We need to pay special attention on the deployment of TREC, hardware architecture of the microcontrollers, and how the constraint space limits affect these above. We next explain each of these aspects.

The recover step in TREC. After computing the clustering results (i.e., partial results for each cluster), the final step is to recover the result matrix from the partial results we just obtained. As shown in Figure 13, the details are

described as follows. We iterate each neuron vector and obtain their *clusterId*. Using this cluster *Id*, we can find the position of the representative vector of this cluster in the stack, thus obtaining the corresponding partial result vector. We retrieve this partial result vector and fill it in the result matrix. After going through all of the neuron vector, we finally manage to reconstruct the result matrix.

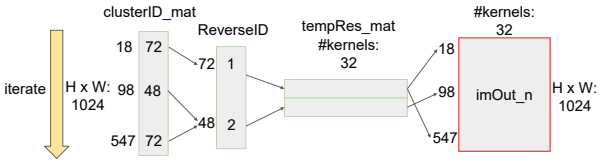


Figure 13: Recovery from the partial results.

Use of SIMD kernels. The Cortex-M series of processors are 32-bit RISC processing cores that are optimized for energy efficiency. They are primarily used as microcontrollers in deeply embedded applications and are part of the ARM Cortex-M architecture [48]. In our work, we enable large DNNs on Cortex-M based systems that offer SIMD instructions, particularly 16-bit Multiply-and-Accumulate (MAC) instructions (e.g. SMLAD), which are extremely beneficial for NN computation.

In the Cortex-M processor, each word, namely 32 bit, can store four integers. Since we use the 16-bit MAC operators, we have to transform the quantized 8-bit data into 16-bit ones. These four 8-bit integer have to be first extended to 16-bit data type, and then reordered to abide with their original order stored in one word.

The matrix multiplication is the most important operator in the entire network. In particular, we have matrix multiplications when performing projection and obtaining partial results in reuse computation. Therefore, we pay special attention on optimizing the performance of matrix multiplication.

The matrix multiplication kernel is implemented focusing on a 2×2 unit at each time, as shown in Figure 17. This allows data reuse while also reducing the overall amount of load instructions. Both operands are of the 16-bit data type, and the accumulation is done with the 32-bit data type. The accumulator is initialized with the corresponding bias value. The MAC instruction __SMLAD is used to execute the computation.

Compared to the original matrix multiplication kernel provided by Cortex-M library, our solution exhibits two major advantages. First, the original kernel requires matrix transpose to conduct multiplication, which does great harm to space locality. Second, the use of SIMD kernels brings about more efficient computations. Experiments show that our solution achieves good results. For instance, for projection in reuse time, we have 1.67× speedup compared to the original matrix multiplication of the Cortex-M library.

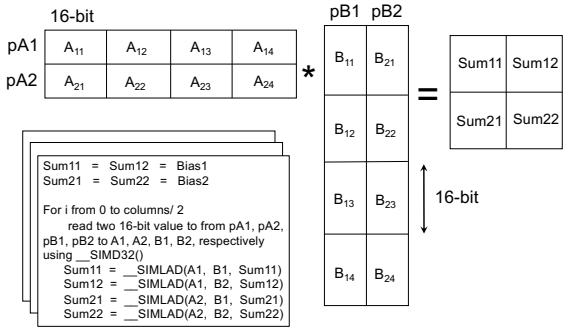


Figure 14: The demonstration of the inner loop of matrix multiplication. Each time two columns and two rows are taken into consideration.

Other hardware characterization. We next describe other hardware characterizations of microcontrollers.

Methodology for placing weights and intermediate activations. For microcontrollers running DNNs, SRAM is used to allocate activation buffers, while Flash memory is used to allocate model weights and biases, as well as the network definition. Alternatively, weights can be stored in SRAM. However, we find that storing weights in Flash only results in a slight increase in end-to-end latency, roughly 1%. This can bring huge benefits because storing weights in Flash saves the limited space usage for activations, which can be placed in SRAM. Storing intermediate activations in Flash, on the other hand, renders a major drop-off in performance (over 20% increase in latency for a layer to store activation maps on the Flash in the current CifarNet setting through an explorative experiment). This is because of the need for block writes, rather than byte writes in Flash. This facet of problem is important as the network grows too large for SRAM and is good for further exploration. This facet of problem includes studies on modifications of kernel implementations, design, and testing of performance models. Specifically, convolution layers write the output data to the Flash memory where the next layer reads and processes data.

Model latency. We next examine the hardware performance of the reuse-centered NN layers. Lai et al. [33] pointed out that, for typical NN layers, latency and energy consumption increases linearly in accord with the operation counts. This model, called latency/energy model, shows that the measured latency can be a function of the number of operations required. In contrast, the reuse-centered layers are able to reduce the total number of operations, thus decreasing the latency of the model.

7. Evaluation

7.1. Experimental Setup

Methodology. To demonstrate the efficacy of TREC in enabling large DNN inference on microcontrollers, we first apply our approach to the entire DNN and measure end-to-end inference performance. Second, we apply TREC and

measure the single-layer acceleration and accuracy for individual layers (specifically the densely-computed layers like convolutional layers). Third, we focus on the space savings of our method. Specifically, how much space is saved from the original DNN models with *offline* methods including pruning, quantization, and layer folding. Since we have shown the TREC space overhead for the three networks in Table 1, we examine the amount of space savings from the two ideas, i.e., *kernel reuse* and *two-step stack*. Fourth, we analyze the performance impact of the number of layers when applying *kernel reuse*. Fifth, we explore TREC performance on sparse inputs. For all the measurements, the SRAM and Flash occupancy is determined using the Mbed compiler (Mbed OS) and the microcontroller latency is measured using the Mbed Timer API.

Platform. DNN inference is performed on the STM32F469I and STM32F746ZG microcontroller featuring SIMD extensions. The STM32F469I platform comes with 324KB SRAM and 2048KB Flash as memory storage while the STM32F746ZG board has 320KB SRAM and 1024KB Flash. We also deploy CMSIS-NN kernels for optimized performance on the Cortex-M4 and Cortex-M7 CPU equipped on the STM32 board [32]. In addition, DNN training is performed on a server equipped by a 20-core 3.60GHz Intel Core i7-12700K CPU with 128GB RAM and an NVIDIA GeForce RTX A6000 GPU with 48 GB memory. We use PyTorch 1.10.1 (open-source software with a BSD license) as our frame for training.

Workloads. We evaluate our approach with the most popular compact DNNs that fit microcontrollers, namely CifarNet [1], ZfNet [59], and SqueezeNet, with and without complex bypass [21]. In all cases, we use the public dataset CIFAR-10 [28] for training and evaluation. All networks are optimized by SGD. The learning rate starts from 0.001 and decreases by 0.1 at 15- epochs intervals. The batch size, weight decay, and momentum are set to 256, 0.9, and 10^{-4}, respectively, and the maximum number of training iterations is set to 100.

7.2. Performance

we compare performance of TREC-equipped DNNs to conventional Convolutional DNNs, *deep reuse* featured DNNs, and LCNNs. Note that after channel pruning, the baseline accuracy of ZfNet is slightly lower than its standard accuracy of 83%, but the model size is 7.6% of the original model, which can well meet the memory constraint of the microcontrollers. Read-only data can be placed in flash memory for microcontroller storage to hold the entire network.

The conventional convolution implementation is from the CMSIS-NN library [32] that achieves the state-of-the-art performance for convolutional neural networks on edge devices. For the datasets, inputs are first converted from floating point representation to 8-bit integers. This implementation is also from the CMSIS-NN library. We compare TREC with LCNN. LCNN stores only a dictionary (which can be regarded as a subset of weight vectors) for reconstructing weights so as to save space and computations. This strategy is actually complementary to TREC. In particular, when we perform convolution in LCNN, the input tensor undergoes a matrix multiplication with dictionary vectors, where our LSH approach can fit in and speed up this matrix multiplication. Through experiments, as shown in Table 2, we find that LCNN experiences slowdown for CifarNet. This is because LCNN requires multiple memory accesses to the dictionary for reconstructing the original weight matrix, which are not suitable for microcontroller applications.

Table 2: Comparison of CMSIS-NN Convolution, LCNN, LCNN with TREC, and TREC for the CifarNet.

	Conv1 (ms)	Conv2 (ms)	Total Latency (ms)
CMSIS Convolution	96.64	57.4	154.69
LCNN	103.51	59.1	162.69
LCNN + TREC	93	49.46	143.18
TREC	53.4	37.76	91.35

For comparison, we also include the results from *deep reuse*, as this state-of-the-art method avoids transient redundancy in DNNs. Given the inherent randomness of *deep reuse*, we present its results in 150-run intervals. Note that *deep reuse* requires storing LSH vectors and index structures on microcontroller memory, which exceeds the memory capacity (denoted as "—" in Table 3 for SqueezeNet and SqueezeNet (Bypass)). For TREC-equipped DNNs, all the space-saving techniques are applied to the model. In Table 3, we have the following findings. First, compared to the conventional convolution (by CMSIS-NN library), TREC achieves 3.61× speedup on average. The same amount of speedups are observed on both microcontrollers, with STM32F746ZG's total end-to-end inference time half of that measured on STM32F469NI. The reason is that Cortex-M7 has a higher clock rate and can issue data loading and ALU instructions at the same time.

Second, TREC experiences only around 0.7% accuracy loss. *Deep reuse* [39] is subject to its inherent randomness, with accuracy loss reaching up to 7.6%. TREC is hence better in terms of stability and accuracy. Since both TREC and *deep reuse* exploit *transient redundancy* elimination, they have comparable inference time. The optimizations in TREC make it however more space efficient. It enables large DNNs (i.e., vanilla SqueezeNet and SqueezeNet with bypass) to run on microcontrollers, while *deep reuse* does not.

Third, when looking into the effects TREC bring to different networks, we find the most speedups in SqueezeNet. This is because there are more computation-intensive con-

Table 3: End-to-end performance and accuracy comparison.

Target(s)	Convolution Methods	CifarNet	ZfNet	SqueezeNet	SqueezeNet (Bypass)
STM32F469NI	Latency (ms)-CMSIS Conv	217.32	3557.32	1639.51	1998.86
	Latency (ms)-*Deep reuse*	154.44	814.03	—	—
	Latency (ms)-TREC	153.92	814.01	327.9	543.71
	Speedup-TREC vs. CMSIS Conv.	1.412×	4.37×	4.98×	3.68×
STM32F746ZG	Latency (ms)-CMSIS Conv	120.62	1758.73	894.16	1152.46
	Latency (ms)-*Deep reuse*	98.44	525.03	—	—
	Latency (ms)-TREC	97.79	524.3	181.49	274.2
	Speedup-TREC vs. CMSIS Conv.	1.23×	3.35×	4.93×	4.20×
Both	Accuracy (%)-CMSIS Conv.	78.2	80.1	83.5	85.6
	Accuracy (%)-*Deep reuse*	73.2 ∼ 76.1	72.5 ∼ 76.6	79.8 ∼ 81.9	80.5 ∼ 83.1
	Accuracy (%)-TREC	76.5	78.9	83	85.3

volution layers in SqueezeNet. The more kernel channels are in a convolution layer, the more redundant computations can be detected and eliminated. We have a more detailed analysis in the following section.

7.3. Single Layer Speedup

We run experiments on each single convolutional layer with a different range of clustering configuration and collect the redundancy ratio. For the purpose of study, we find the configurations that can reduce the maximum amount of computations without threatening inference accuracy. Latencies are collected from the STM32F469NI board.

We list in Table 4 the single-layer speedups and accuracy losses attained by substituting TREC for the traditional convolution. The following observations have been made. First, TREC finds and removes approximately 96.22% of the *transient redundancy* indicated by r_t. This is where the speedup comes from.

Second, with an average speedup of 4.28×, TREC significantly improves the performance of each convolution layer. Specifically, TREC speeds up SqueezeNet expand layers by up to 18.66×, showing that computation-intensive layers can particularly benefit from TREC. However, due to the additional cost from clustering, the speedup is smaller than what is suggested by the ratio of *transient redundancy* that TREC reduces.

Third, there is a limited and mixed impact of TREC on accuracy. It may result in a 0.8% accuracy decrease when applied to certain layers, but it may also result in a modest accuracy improvement when applied to some other layers. Overall, TREC keeps the accuracy much better than *deep reuse* does (around 3%-7% drop). The single-layer results show that TREC can operate at maximum efficiency, achieving notable speedups with the least amount of accuracy loss.

7.4. Space Saving Analysis

There are static space and dynamic space required for storing models. Specifically, static space refers to the

Table 4: Single-layer performance benefits. $Conf.$ means configuration, L is the width of sub-matrices, H is the number of hash functions. K is the kernel size and M is the number of kernel channels. r_t represents the redundancy ratio, and ΔAcc is the accuracy difference between TREC and standard convolution.

Network	ConvLayer	K	M	Conf. L	Conf. H	r_t	Speedup	ΔAcc
CifarNet	Conv1	75	64	15	5	0.943	1.81×	-0.0132
	Conv2	1600	64	10	10	0.795	1.52×	-0.0077
	Avg.					0.869	1.66×	-0.0105
ZfNet	Conv1	147	96	147	5	0.999	1.22×	-0.0011
	Conv2	2400	256	300	5	0.997	4.69×	-0.0037
	Conv3	2304	384	384	5	0.988	4.72×	-0.0076
	Conv4	3456	384	432	5	0.998	6.23×	-0.0105
	Conv5	3456	256	288	5	0.984	5.58×	-0.0068
	Avg.					0.994	4.49×	-0.0059
SqueezeNet	Conv1	27	96	9	5	0.997	1.27×	-0.0122
	Fire2.squeeze.conv	96	16	96	4	0.993	1.07×	-0.0144
	Fire2.expand_1x1.conv	16	64	8	5	0.991	1.41×	-0.0023
	Fire2.expand_3x3.conv	144	64	48	5	0.990	5.90×	-0.0002
	Bypass1	96	128	32	5	0.994	2.45×	-0.0071
	Fire3.squeeze.conv	128	16	64	5	0.988	1.07×	0.0051
	Fire3.expand_1x1.conv	16	64	8	5	0.992	2.30×	0.0033
	Fire3.expand_3x3.conv	144	64	24	5	0.988	5.85×	0.0013
	Fire4.squeeze.conv	128	32	64	5	0.990	1.00×	0.004
	Fire4.expand_1x1.conv	128	128	4	8	0.988	1.45×	0.0037
	Fire4.expand_3x3.conv	32	128	16	5	0.988	6.36×	-0.0012
	Bypass2	288	256	48	5	0.993	2.53×	0.0012
	Fire5.squeeze.conv	256	32	64	4	0.982	1.84×	0.0036
	Fire5.expand_1x1.conv	32	128	16	5	0.953	4.51×	-0.0016
	Fire5.expand_3x3.conv	288	128	48	5	0.954	11.83×	-0.0001
	Fire6.squeeze.conv	256	48	128	5	0.972	1.01×	0.0018
	Fire6.expand_1x1.conv	256	192	32	5	0.963	4.40×	0.0071
	Fire6.expand_3x3.conv	48	192	32	5	0.953	12.50×	-0.0002
	Bypass3	432	384	72	5	0.977	5.29×	-0.0021
	Fire7.squeeze.conv	384	48	48	5	0.965	1.71×	-0.0003
	Fire7.expand_1x1.conv	48	192	24	5	0.957	4.00×	0.0046
	Fire7.expand_3x3.conv	432	192	48	5	0.953	9.36×	0.0025
	Fire8.squeeze.conv	192	64	64	4	0.969	1.03×	0.0003
	Fire8.expand_1x1.conv	384	256	128	5	0.965	2.15×	-0.0012
	Fire8.expand_3x3.conv	64	256	16	5	0.955	7.57×	0.0022
	Bypass4	576	512	64	5	0.961	1.12×	0.0045
	Fire9.squeeze.conv	512	64	128	4	0.894	1.01×	0.0078
	Fire9.expand_1x1.conv	64	256	16	5	0.839	1.53×	0.0019
	Fire9.expand_3x3.conv	576	256	96	5	0.841	11.64×	0.0022
	Conv10	512	10	4	5	0.944	1.29×	-0.0057
	Avg. (w/ bypass)					0.963	3.88×	0.0003
	Avg. (w/o bypass)					0.963	4.89×	-0.0006

space for weights and biases (from the original model). Dynamic space contains TREC space, namely, *Hash* matrix and index structures. The baseline size for original static and dynamic space occupancy is detailed in Table 5.

Table 5: Space savings by different techniques.

	Technique	CifarNet	ZfNet	SqueezeNet
Static Space	Baseline Size	420 KB	4 MB	5 MB
	Channel Pruning	45.1%	92.4%	90.8%
	CP + Q	86.8%	97.9%	97.4%
	CP + Q + LF	86.8%	98.9%	98.2%
Dynamic Space	Baseline Size	20 KB	40KB	56 KB
	Kernel Reuse	41.2%	68 .2%	71.9%
	KR + TSS	60.2%	73.4%	77.3%

CP: Channel Pruning, Q: Quantization, LF: Layer Foldings, KR: Kernel Reuse, TSS: Two-Step Stack

For static space, we apply existing techniques to remove *lasting redundancy*. For dynamic space, we use *kernel reuse* and *two-step stack* to eliminate *transient redundancy*. Table 5 summarizes how much space is saved for the original model size and TREC size, respectively. For static space, since there is no Batch norm layer in CifarNet Overall, the original model size witnesses an average of over 90% reduction as a result of these *lasting redundancy elimination* based methods. For dynamic space, we report the space reduction for *kernel reuse* with configurations where there is virtually no accuracy loss ($< 0.1\%$), and provide a detailed analysis in Section 7.5. The dynamic space reduction exceeds 70% when we combine *kernel reuse* and *two-step stack*.

7.5. Detailed Analysis

Kernel reuse for reaching accuracy stability. *Kernel reuse* embeds the *Hash* matrix in the weight matrix W. Figure 15 shows how *kernel reuse* influences SqueezeNet in terms of both accuracy and space occupancy on the STM32F7 board. The x axis denotes the number of layers we apply *kernel reuse* for SqueezeNet. We first look at the accuracy. For comparison, we also include the accuracy of *deep reuse* at 150 runs. *Deep reuse* has random LSH vectors, and its accuracy varies between $80\% \sim 83\%$. Accordingly, the two green lines show the accuracy range that *deep reuse* produces. In contrast, *kernel reuse* produces stable accuracy results up to 85.6%. Although there is accuracy loss as x increases, the network accuracy drops slowly and remains high ($> 84\%$) for $x < 16$.

Second, *kernel reuse* brings huge space benefits. For example, when *kernel reuse* is in five layers, namely, $x = 5$, the space overhead is cut by over 50% with accuracy loss $\delta < 0.5\%$. That is to say, for large DNNs, applying *kernel reuse* to a small number of layers can dramatically reduce the total space overhead with marginal accuracy loss.

Sparse inputs. We apply TREC to sparse input and achieve speedups. For Cifarnet, we detail the performance

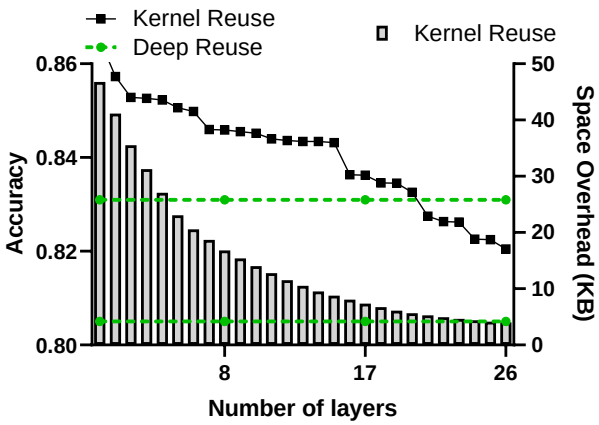


Figure 15: Influence of accuracy and space overhead from the number of layers applying kernel reuse.

for SparseConv and TREC in Table 6. Input sparsity is set to 0.1 and batch size is 10. The speedup are 3.38× and 4.83× for the two convolution layers, respectively.

Table 6: Performance for sparse input matrix.

ConvLayer	Output Channels	Latency (ms) SparseConv	Latency (ms) TREC	Speedup
Conv1	32	21.98	6.51	3.38x
Conv2	64	42.96	8.89	4.83x

8. Conclusion

In this work, we demonstrated that large DNNs can be deployed on resource-constrained microcontrollers and executed efficiently by the introduction of space-efficient TREC. It takes advantages of both computation reduction and space savings that make the best ability of the resource utilization of microcontrollers. As a result, the approach maximizes the performance benefits and obtain stable accuracy results. In the evaluation, our solution achieves 3.5×-5× speedups with virtually no accuracy loss.

References

[1] "CifarNet," http://places.csail.mit.edu/deepscene/small-projects/TRN-pytorch-pose/model_zoo/models/slim/nets/cifarnet.py, 2020.

[2] Anonymous, "Trec: Transient redundancy elimination-based convolution (technical report)," 2022. [Online]. Available: https://drive.google.com/file/d/1ctveFt9gv4nNmx9JPKC1jrEL6vjLBj_t/view

[3] P. Bajcsy and M. Majurski, "Baseline pruning-based approach to trojan detection in neural networks," *arXiv preprint arXiv:2101.12016*, 2021.

[4] C. Banbury, C. Zhou, I. Fedorov, R. Matas, U. Thakker, D. Gope, V. Janapa Reddi, M. Mattina, and P. Whatmough, "Micronets: Neural network architectures for deploying tinyml applications on commodity microcontrollers," *Proceedings of Machine Learning and Systems*, vol. 3, pp. 517–532, 2021.

[5] J. Benito-Picazo, E. Domínguez, E. J. Palomo, E. López-Rubio, and J. M. Ortiz-de Lazcano-Lobato, "Deep learning-based anomalous object detection system powered by microcontroller for ptz cameras," in *2018 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 2018, pp. 1–7.

[6] N. Bhave, A. Dhagavkar, K. Dhande, M. Bana, and J. Joshi, "Smart signal–adaptive traffic signal control using reinforcement learning and object detection," in *2019 Third International conference on I-SMAC (IoT in Social, Mobile, Analytics and Cloud)(I-SMAC)*. IEEE, 2019, pp. 624–628.

[7] D. E. Bolanakis, "A survey of research in microcontroller education," *IEEE Revista Iberoamericana de Tecnologias del Aprendizaje*, vol. 14, no. 2, pp. 50–57, 2019.

[8] G. Cerutti, R. Andri, L. Cavigelli, E. Farella, M. Magno, and L. Benini, "Sound event detection with binary neural networks on tightly power-constrained iot devices," in *Proceedings of the ACM/IEEE International Symposium on Low Power Electronics and Design*, 2020, pp. 19–24.

[9] B. Chen, Z. Liu, B. Peng, Z. Xu, J. L. Li, T. Dao, Z. Song, A. Shrivastava, and C. Re, "{MONGOOSE}: A learnable {lsh} framework for efficient neural network training," in *International Conference on Learning Representations*, 2021. [Online]. Available: https://openreview.net/forum?id=wWK7yXkULyh

[10] A. Company, "Cortex®-m4 technical reference manual," 2010. [Online]. Available: https://users.ece.utexas.edu/~valvano/EE345L/Labs/Fall2011/CortexM4_TRM_r0p1.pdf

[11] R. David, J. Duke, A. Jain, V. Janapa Reddi, N. Jeffries, J. Li, N. Kreeger, I. Nappier, M. Natraj, T. Wang *et al.*, "Tensorflow lite micro: Embedded machine learning for tinyml systems," *Proceedings of Machine Learning and Systems*, vol. 3, pp. 800–811, 2021.

[12] A. E. Eshratifar, A. Esmaili, and M. Pedram, "Bottlenet: A deep learning architecture for intelligent mobile cloud computing services," in *2019 IEEE/ACM International Symposium on Low Power Electronics and Design (ISLPED)*. IEEE, 2019, pp. 1–6.

[13] D. Farren, T. Pham, and M. Alban-Hidalgo, "Low latency anomaly detection and bayesian network prediction of anomaly likelihood," *arXiv preprint arXiv:1611.03898*, 2016.

[14] I. Fedorov, R. P. Adams, M. Mattina, and P. Whatmough, "Sparse: Sparse architecture search for cnns on resource-constrained microcontrollers," *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[15] I. Fedorov, R. P. Adams, M. Mattina, and P. Whatmough, "Sparse: Sparse architecture search for cnns on resource-constrained microcontrollers," *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[16] B. Graham, M. Engelcke, and L. Van Der Maaten, "3d semantic segmentation with submanifold sparse convolutional networks," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 9224–9232.

[17] C. Gupta, A. S. Suggala, A. Goyal, H. V. Simhadri, B. Paranjape, A. Kumar, S. Goyal, R. Udupa, M. Varma, and P. Jain, "Protonn: Compressed and accurate knn for resource-scarce devices," in *International Conference on Machine Learning*. PMLR, 2017, pp. 1331–1340.

[18] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding," *arXiv preprint arXiv:1510.00149*, 2015.

[19] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network (2015)," *arXiv preprint arXiv:1503.02531*, vol. 2, 2015.

[20] F. N. Iandola, S. Han, M. W. Moskewicz, K. Ashraf, W. J. Dally, and K. Keutzer, "Squeezenet: Alexnet-level accuracy with 50x fewer parameters and< 0.5 mb model size," *arXiv preprint arXiv:1602.07360*, 2016.

[21] F. N. Iandola, S. Han, M. W. Moskewicz, K. Ashraf, W. J. Dally, and K. Keutzer, "SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and< 0.5 MB model size," *arXiv preprint arXiv:1602.07360*, 2016.

[22] S. Jacob, V. G. Menon, F. Al-Turjman, P. Vinoj, and L. Mostarda, "Artificial muscle intelligence system with deep learning for post-stroke assistance and rehabilitation," *Ieee Access*, vol. 7, pp. 133 463–133 473, 2019.

[23] J. Kaivo-oja, "Weak signals analysis, knowledge management theory and systemic socio-cultural transitions," *Futures*, vol. 44, no. 3, pp. 206–217, 2012.

[24] K. Kaur, S. Garg, G. S. Aujla, N. Kumar, J. J. Rodrigues, and M. Guizani, "Edge computing in the industrial internet of things environment: Software-defined-networks-based edge-cloud interplay," *IEEE communications magazine*, vol. 56, no. 2, pp. 44–51, 2018.

[25] D. Kim, K. Park, Y. Park, and J.-H. Ahn, "Willingness to provide personal information: Perspective of privacy calculus in iot services," *Computers in Human Behavior*, vol. 92, pp. 273–281, 2019.

[26] A. Kolesau and D. Šešok, "Voice activation systems for embedded devices: Systematic literature review," *Informatica*, vol. 31, no. 1, pp. 65–88, 2020.

[27] A. Krizhevsky, G. Hinton *et al.*, "Learning multiple layers of features from tiny images," 2009.

[28] A. Krizhevsky, G. Hinton *et al.*, "Learning multiple layers of features from tiny images," 2009.

[29] A. Kumar, S. Goyal, and M. Varma, "Resource-efficient machine learning in 2 kb ram for the internet of things," in *International Conference on Machine Learning*. PMLR, 2017, pp. 1935–1944.

[30] L. Lai and N. Suda, "Enabling deep learning at the lot edge," in *2018 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, 2018, pp. 1–6.

[31] L. Lai, N. Suda, and V. Chandra, "Deep convolutional neural network inference with floating-point weights and fixed-point activations," *arXiv preprint arXiv:1703.03073*, 2017.

[32] L. Lai, N. Suda, and V. Chandra, "Cmsis-nn: Efficient neural network kernels for arm cortex-m cpus," *arXiv preprint arXiv:1801.06601*, 2018.

[33] L. Lai, N. Suda, and V. Chandra, "Not all ops are created equal!" *arXiv preprint arXiv:1801.04326*, 2018.

[34] X. Li, J. Guivant, N. Kwok, Y. Xu, R. Li, and H. Wu, "Three-dimensional backbone network for 3d object detection in traffic scenes," *arXiv preprint arXiv:1901.08373*, 2019.

[35] A. Massa, D. Marcantonio, X. Chen, M. Li, and M. Salucci, "Dnns as applied to electromagnetics, antennas, and propagation—a review," *IEEE Antennas and Wireless Propagation Letters*, vol. 18, no. 11, pp. 2225–2229, 2019.

[36] S. Mittermaier, L. Kürzinger, B. Waschneck, and G. Rigoll, "Small-footprint keyword spotting on raw audio data with sinc-convolutions," in *ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2020, pp. 7454–7458.

[37] M. V. Ngo, H. Chaouchi, T. Luo, and T. Q. Quek, "Adaptive anomaly detection for iot data in hierarchical edge computing," *arXiv preprint arXiv:2001.03314*, 2020.

[38] L. Ning and X. Shen, "Deep reuse: streamline cnn inference on the fly via coarse-grained computation reuse," in *Proceedings of the ACM International Conference on Supercomputing*, 2019, pp. 438–448.

[39] L. Ning and X. Shen, "Deep Reuse: streamline CNN inference on the fly via coarse-grained computation reuse," in *Proceedings of the ACM International Conference on Supercomputing*, 2019, pp. 438–448.

[40] W. Niu, X. Ma, S. Lin, S. Wang, X. Qian, X. Lin, Y. Wang, and B. Ren, "Patdnn: Achieving real-time dnn execution on mobile devices with pattern-based weight pruning," in *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 907–922. [Online]. Available: https://doi.org/10.1145/3373376.3378534

[41] N. P. Novani, M. H. Hersyah, and R. Hamdanu, "Electrical household appliances control using voice command based on microcontroller," in *2020 International Conference on Information Technology Systems and Innovation (ICITSI)*. IEEE, 2020, pp. 288–293.

[42] M. Paganini and J. Forde, "Streamlining tensor and network pruning in pytorch," *arXiv preprint arXiv:2004.13770*, 2020.

[43] Z. Qin, Z. Zhang, X. Chen, C. Wang, and Y. Peng, "Fd-mobilenet: Improved mobilenet with a fast downsampling strategy," in *2018 25th IEEE International Conference on Image Processing (ICIP)*. IEEE, 2018, pp. 1363–1367.

[44] M. Riera, J.-M. Arnau, and A. Gonzalez, "Computation reuse in dnns by exploiting input similarity," in *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*, 2018, pp. 57–68.

[45] M. Rusci, A. Capotondi, and L. Benini, "Memory-driven mixed low precision quantization for enabling deep network inference on microcontrollers," *Proceedings of Machine Learning and Systems*, vol. 2, pp. 326–335, 2020.

[46] F. Salewski and S. Kowalewski, "Hardware/software design considerations for automotive embedded systems," *IEEE Transactions on Industrial Informatics*, vol. 4, no. 3, pp. 156–163, 2008.

[47] J. Shao and J. Zhang, "Bottlenet++: An end-to-end approach for feature compression in device-edge co-inference systems," in *2020 IEEE International Conference on Communications Workshops (ICC Workshops)*. IEEE, 2020, pp. 1–6.

[48] P. Sharma and D. Kamthania, "Intelligent object detection and avoidance system," in *International Conference on Transforming IDEAS (Inter-Disciplinary Exchanges, Analysis, and Search) into Viable Solutions*, 2019, pp. 342–351.

[49] S. Soro, "Tinyml for ubiquitous edge ai," *arXiv preprint arXiv:2102.01255*, 2021.

[50] S. R. Sridhara, "Ultra-low power microcontrollers for portable, wearable, and implantable medical electronics," in *16th Asia and South Pacific Design Automation Conference (ASP-DAC 2011)*. IEEE, 2011, pp. 556–560.

[51] H. Teraoka, F. Nakahara, and K. Kurosawa, "Incremental update method for vehicle microcontrollers," in *2017 IEEE 6th Global Conference on Consumer Electronics (GCCE)*. IEEE, 2017, pp. 1–2.

[52] C.-B. Tzeng, "Vibration detection and analysis of wind turbine based on a wireless embedded microcontroller system," in *2018 IEEE International Conference on Applied System Invention (ICASI)*. IEEE, 2018, pp. 133–136.

[53] R. Wu, F. Zhang, Z. Zheng, X. Du, and X. Shen, "Exploring deep reuse in winograd cnn inference," in *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, 2021, pp. 483–484.

[54] Y. Yan, Y. Mao, and B. Li, "Second: Sparsely embedded convolutional detection," *Sensors*, vol. 18, no. 10, p. 3337, 2018.

[55] H. Yeo, Y. Jung, J. Kim, J. Shin, and D. Han, "Neural adaptive content-aware internet video delivery," in *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, 2018, pp. 645–661.

[56] J. Yi, Y. Tan, Z. Ang, and S. Panda, "Microcontroller based voice-activated powered wheelchair control," in *Proceedings of the 1st international convention on Rehabilitation engineering & assistive technology: in conjunction with 1st Tan Tock Seng Hospital Neurorehabilitation Meeting*, 2007, pp. 67–72.

[57] Y. Yu, Z. Yang, Y. You, and W. Shan, "Fassnet: fast apnea syndrome screening neural network based on single-lead electrocardiogram for wearable devices," *Physiological Measurement*, vol. 42, no. 8, p. 085005, 2021.

[58] J. Yuan, K. K. Tan, T. H. Lee, and G. C. H. Koh, "Power-efficient interrupt-driven algorithms for fall detection and classification of activities of daily living," *IEEE Sensors Journal*, vol. 15, no. 3, pp. 1377–1387, 2014.

[59] M. D. Zeiler and R. Fergus, "Visualizing and understanding convolutional networks," in *European conference on computer vision*. Springer, 2014, pp. 818–833.

[60] Y. Zhang, N. Suda, L. Lai, and V. Chandra, "Hello edge: Keyword spotting on microcontrollers," *arXiv preprint arXiv:1711.07128*, 2017.

A. Sparse Input Matrices: Converting Sparse Convolution to Dense Convolution

As introduced in Section 4.5 , TREC is also compatible with sparse input matrices through *sparse convolution*.

As Figure 16 illustrates, we have a 5×5 image with three channels. All the pixels are (0,0,0) except three points P_1, P_2, and P_3. The convolution kernel of sparse convolution is the same as traditional convolution. Each kernel size is 3×3. We can distinguish three filters according to the light and dark colors. In this example, we use convolution with the stride of one and padding of zero. There are three filters. Accordingly, the convolution will produce three 3×3 output activation maps (which are not displayed in the figures).

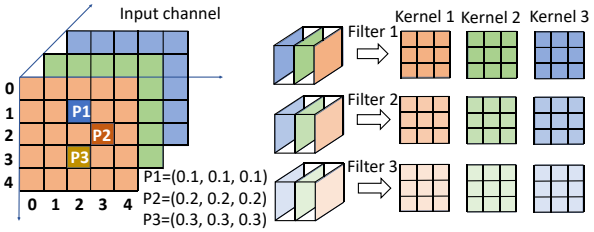


Figure 16: Illustration of input model and kernels.

We label the nine kernels as $kernel[k]_{i,j}, 1 \leq k \leq 9$. Considering the specific convolution, for each of the 3×3 kernel position $(i, j), 0 \leq i, j < 3$, we collect all the atomic operations for P_1, P_2, and P_3 with respect to $kernel[k]_{i,j}, 1 \leq k \leq 9$. For the kernel position $(1, 1)$, namely, F_4, we collect P_1, P_2, and P_3 as a reorganized matrix and compute them with F_4 together, as shown in Figure 17. For the kernel position F_0, the only generated operations are with P_1. Likewise, for the kernel position F_4, the generated operations are with P_1, P_2, and P_3.

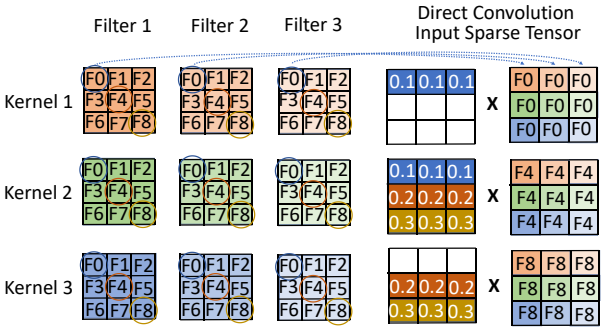


Figure 17: Direct convolution through dense matrix multiplication.

Since there are three channels and three filters, we have a total of nine kernels. Therefore, for each of the kernel position $(i, j), 0 \leq i, j < 3$, the elements corresponding to (i, j) can form a 3×3 matrix. Accordingly, the direct convolution is constructed as a matrix multiplication. To this end, we can convert sparse convolution into a dense matrix multiplication through computing reorganization. This matrix multiplication can be accelerated by incorporating LSH-based clustering.